

STRIDE

Smart Trading & Retail Intelligence for Digital Economies

Product & Technical Documentation

Internet of Agents Hackathon 2026

A multi-agent AI system that learns your preferences,
scrapes real products, enforces agentic guardrails,
and pays autonomously via x402 protocol.

Team

Swati Chauhan — Auth, Payment, x402, Dashboard, Guardrails
Rohit Mujumdar — RL, Reward, Shopping Agent, Scraping, Memory

Tech Stack

Claude Sonnet 4.6 • Thompson Sampling • x402/USDC • Base Sepolia
Jina.ai • Streamlit • FastAPI • Python 3.12

Problem & Solution

Why autonomous shopping agents don't exist yet

THE PROBLEM

AI assistants can recommend products but cannot buy them. The gap between "here's what I found" and "I bought it for you" requires solving 5 hard problems:

- Discovery: Agents can't browse real websites — limited to APIs or hardcoded catalogs
- Learning: Each session starts from scratch — no memory of what worked before
- Payment: No standard protocol for machines to pay machines autonomously
- Trust: If an agent can spend your money, who controls what it's allowed to do?
- Collaboration: Agents from different teams can't work together — no shared standards

OUR SOLUTION: STRIDE

- Discovery: Live scraping of Zappos, StockX, GOAT, 6pm via Jina.ai — real products, real prices
- Learning: Thompson Sampling RL across 4 dimensions — measurably smarter over runs
- Payment: x402 protocol — real USDC on Base Sepolia, verifiable on BaseScan
- Trust: 4-layer guardrail system — preferences, auth, payment checks, protocol verification
- Collaboration: Agent UX endpoints — any external agent can discover and interact via HTTP

KEY INSIGHT

An AI shopping agent needs the same infrastructure humans have: identity (who am I shopping for?), trust (what am I allowed to do?), learning (what worked last time?), payment (how do I pay?), guardrails (what should I never do?).

System Architecture

6 agents connected by structured protocols

THE 6-AGENT PIPELINE

Each run: Memory → RL Bandit → Auth → Shopping Agent → Payment → Reflexion → Memory

Agent 1: Memory

memory_agent.py (Rohit)
Reads context_graph.json
User prefs, history, insights
Single source of truth

Agent 2: RL Bandit

rl_bandit.py (Rohit)
Thompson Sampling
4 bandits: site, query, filter, abandon. Persists across sessions.

Agent 3: Auth

auth_agent.py (Swati)
Issues scoped credential
5 verification checks
Human approval gate (runs 1-3)

Agent 4: Shopping

shopping_agent.py (Rohit+Swati)
Claude Sonnet 4.6 ReAct loop
7 actions incl NEED_APPROVAL
Guardrails in system prompt

Agent 5: Payment

payment_agent.py (Swati)
Real x402 on Base Sepolia
USDC via EIP-3009
Simulation fallback mode

Agent 6: Reflexion

run_loop.py (Rohit)
Claude writes 1-sentence lesson
Fed into next run's prompt
Human-readable learning

AGENT-TO-AGENT COMMUNICATION

Not just sequential — Shopping Agent queries Memory Agent mid-run:

Shopping: ASK_MEMORY "which site had best Yeezy prices?"

Memory: "StockX had reward 99, GOAT had 51"

Shopping: "I'll try StockX first."

ORCHESTRATION

run_loop.py is a simple Python for-loop calling each agent in sequence.

No LangGraph, no LangChain, no framework. Each agent is a pure function that takes a dict and returns a dict — LangGraph-ready by design.

Complexity is in the agents, not the wiring.

Reinforcement Learning

Thompson Sampling — how the agent learns

THE EXPLORE-EXPLOIT PROBLEM

Should the agent try a new site (explore) or stick with what worked (exploit)?

Thompson Sampling solves this with Bayesian probability.

HOW IT WORKS

- Each arm (e.g., site=zappos) has a Beta(alpha, beta) distribution
- Alpha = accumulated successes + 1 (prior). Beta = accumulated failures + 1
- To select: sample from each arm's Beta distribution, pick the highest
- After reward: normalize to [0,1], add to alpha (success) and beta (failure)
- Over time: successful arms get higher alpha → sampled more → exploitation
- But Beta distributions always have variance → occasionally tries low arms → exploration

4 INDEPENDENT BANDITS

- Site (6 arms): zappos, 6pm, stockx, goat, nike, amazon
- Query Style (3 arms): broad, moderate, specific
- Filter Strategy (4 arms): price_first, brand_first, rating_first, size_first
- Abandon Threshold (3 arms): after_1_fail, after_2_fails, after_3_fails

REWARD FUNCTION (reward.py)

- Purchase completed: +40 | Full match: +25 | Under budget: +15 | Fast delivery: +10
- Efficiency bonus (4 steps or fewer): +15 | Checkout reached: +10 | Partial match: +5
- Per step penalty: -4 each | Captcha: -15 | Out of stock: -20 | Over budget: -10
- Range: perfect run ~99 pts, failed run ~-55 pts. Wide spread makes learning visible.
- MAX_REWARD = 115 for normalization. Bandit state persists to bandit_state.json.

x402 Protocol

Machine-to-machine payments over HTTP

WHAT IS x402?

HTTP 402 ("Payment Required") was reserved in 1999 but never used.

Coinbase created x402 to implement it — enabling AI agents to pay for services using stablecoins (USDC) over standard HTTP.

THE 7-STEP FLOW

- Step 1: Payment Agent sends POST /purchase to x402 Server
- Step 2: Server returns HTTP 402 + X-PAYMENT-REQUIRED header (price, token, recipient)
- Step 3: x402 SDK selects the matching payment scheme (EVM/Base Sepolia)
- Step 4: EthAccountSigner signs USDC transferWithAuthorization (EIP-3009)
- Step 5: SDK retries request with X-PAYMENT header (signed proof)
- Step 6: Server sends to facilitator (x402.org) for on-chain settlement
- Step 7: Facilitator verifies, USDC settles on Base Sepolia, server returns 200 OK

OUR IMPLEMENTATION

- Client: payment_agent.py — x402 SDK + EthAccountSigner, private key from .env
- Server: x402_server.py — FastAPI + PaymentMiddlewareASGI on port 4021
- Network: Base Sepolia (eip155:84532) — Coinbase L2 testnet
- Token: USDC at 0x036CbD53842c5426634e7929541eC2318f3dCF7e
- Facilitator: x402.org/facilitator — verifies and settles on-chain
- Wallet: eth_account.Account.create() — funded with free test USDC from Circle faucet

WHY x402 MATTERS

x402 is an open standard. Any agent that speaks HTTP can pay any

x402-enabled server. No API keys, no OAuth — the wallet IS the credential.

When stores adopt x402, our agent connects directly — zero code change.

Agentic Guardrails

4 layers of protection before any purchase

WHY GUARDRAILS MATTER

An autonomous agent that can spend money needs constraints. Without guardrails, it might buy the wrong product, overspend, or ignore user's instructions.

STRIDE uses defense-in-depth: 4 independent layers, each catching different failures.

LAYER 1: PREFERENCE GUARDRAILS (`shopping_agent.py`)

- `color`: "REQUIRED COLOR: blue — only buy this color"
- `specific_product`: "ONLY buy Nike x Tiffany AF1. Do not substitute."
- `custom_instructions`: User's exact words injected into Claude's system prompt
- `excluded_brands`: "Never buy from these brands"
- `NEED_APPROVAL` action: flags non-matching products instead of buying

LAYER 2: AUTH GUARDRAILS (`auth_agent.py`)

- Session expiry (1 hour) • Site whitelist (6 stores) • Hard spend cap
- Budget enforcement • Human approval gate (first 3 runs)

LAYER 3: PAYMENT GUARDRAILS (`payment_agent.py`)

- Auth clearance required • Double-checks spend cap • Exploration mode (no payment until final run)

LAYER 4: PROTOCOL GUARDRAILS (`x402` + `blockchain`)

- `x402` middleware verifies payment signature • Facilitator validates on-chain
- Transaction irreversible but transparent • Verifiable on BaseScan

GUARDRAILS IN ACTION

User: "I want Tiffany Nike AF1, ask before buying anything else"

Agent finds Nike Pegasus → Layer 1: `NEED_APPROVAL` (not the requested product)

Agent finds Tiffany AF1 over budget → Layer 2: blocked (exceeds budget)

Agent finds Tiffany AF1 on bad site → Layer 2: blocked (site not in whitelist)

Agent finds Tiffany AF1 on StockX at right price → All 4 layers pass → `x402` pays

Live Scraping & Agent UX

Real products + endpoints for the Internet of Agents

LIVE WEB SCRAPING (live_scraper.py)

Jina.ai Reader fetches pages as markdown. Custom regex parsers extract products.

Claude fallback if regex fails. Search URLs built dynamically from user brands.

- Zappos (LIVE): regex captures name, price, rating, URL from markdown links
- 6pm (LIVE): same parser as Zappos — discount inventory
- StockX (LIVE): regex captures from "[Product Lowest Ask price](url)" format
- GOAT (LIVE): regex captures from nested markdown image+link format
- Nike (SIMULATED): JS-heavy, falls back to catalog | Amazon (SIMULATED): bot-blocked

AGENT UX ENDPOINTS (x402_server.py)

Traditional UX = buttons for humans. Agent UX = structured endpoints for AI agents.

- GET /agent-profile — user preferences, trust rules, RL weights, lessons, payment info
- GET /agent-capabilities — every endpoint with description and input/output schema
- GET /agent-history — past runs with rewards, strategies, products, summary stats
- POST /purchase — x402-protected, any agent with USDC wallet can pay

CONTEXT GRAPH SCHEMA (context_graph.json)

The single source of truth. Every agent reads from it. Onboarding writes to it.

- preferences: brands, size, budget, max_delivery_days, style + guardrails (color, specific_product, custom_instructions, excluded_brands)
- trust_rules: max_autonomous_spend, approved_categories, require_approval_first_n_runs
- history: every past run (strategy, reward, outcome)
- learned_insights: reflexion lessons from Claude
- rl_weights: current Thompson Sampling bandit weights per dimension

INTERNET OF AGENTS ALIGNMENT

- 6 agents collaborating via structured protocols
- x402 machine-to-machine payments — open standard
- Agent UX endpoints — any agent can discover and interact
- Agentic guardrails — user-defined rules that constrain agents
- Learning compounds — bandit state persists, reflexion lessons feed next run

Demo Script

2.5 minutes — what to show and when

THE DEMO FLOW

- [0:00 - 0:30] ONBOARDING: Type "I want Nike running shoes, size 10, budget 120, need them in 2 days." Show Claude extracting preferences. "Profile saved!"
- [0:30 - 0:45] GUARDRAILS: Point to sidebar showing brands, size, budget, color, custom instructions all captured from natural language.
- [0:45 - 0:50] LOAD RESULTS: Click "Load from run_results.json" — instant, no API calls.
- [0:50 - 1:10] REWARD CURVE: "Run 1 scored -24. By Run 8, it scored 99. The agent taught itself which site and strategy works best. That's Thompson Sampling."
- [1:10 - 1:40] REASONING TRACE: Compare Run 1 (8 steps, wrong site, abandoned) vs Run 8 (4 steps, perfect match). "No human told it which site was better."
- [1:40 - 1:55] PURCHASES: "Real products scraped from real websites. Click this — it goes to the actual product page on StockX."
- [1:55 - 2:10] PAYMENT: "Real x402 USDC transaction on Base Sepolia." Click BaseScan link. "Zero human in the loop."
- [2:10 - 2:30] AGENT UX: Click "Test /agent-profile". "Any agent on the internet can call this and instantly know how to shop for this user."

IF JUDGES ASK...

- "Can you run it live?" → Wait 60s (rate limit), click Run Agent Live with slider at 3.
- "What if I want something specific?" → Go to onboarding, type "I want Jordan 4 in black under 300, ask before buying anything else." Show guardrails.
- "Is this LangGraph?" → "No — simpler. Pure functions, simple loop. Complexity is in agents, not wiring. LangGraph-ready by design."
- "Is the payment real?" → "Yes — real USDC on Base Sepolia testnet. Click the BaseScan link. To go mainnet: change one config line."

ONE-LINER FOR JUDGES

"We built a network of 6 AI agents that scrape real websites, learn via reinforcement learning, respect user-defined guardrails, and pay with real cryptocurrency — all autonomously. This is what the internet looks like when agents are the users."